INSTRUCTOR SESSION GUIDE

Day 07

Strings & String Methods

■ INTERMEDIATE PHASE | Day 7 of 30 | 2 Hours

Session Goal	Master Python string manipulation — formatting, methods, and regex basics
Prerequisites	Days 1–6 complete: variables, control flow, loops, functions, data structures
Students Produce	String formatter, text analyser, regex pattern matcher
Next Session	Day 08 — File I/O (open, read, write, CSV, JSON)

TODAY'S CONCEPT MAP

String Formatting	String Methods	Regex Basics	Mini Project

■ SESSION TIMELINE — 2-HOUR MINUTE-BY-MINUTE PLAN

Time	Duration	Activity	Key Points
0:00–0:10	10 min	Warm-up & Recap	Quiz: Day 6 data structures — list vs dict vs set
0:10–0:25	15 min	String Formatting	%, .format(), f-strings — live comparison demo
0:25–0:45	20 min	Core String Methods	split/join/strip/replace/upper/lower/find/count
0:45–1:00	15 min	Guided Exercise 1	Students build a text cleaner function
1:00–1:05	5 min	Break	
1:05–1:20	15 min	Regex Basics	re.search, re.match, re.findall with live patterns
1:20–1:35	15 min	Guided Exercise 2	Email / phone validator with regex
1:35–1:50	15 min	Mini-Project	Student contact-card formatter
1:50–2:00	10 min	Wrap-up & Preview	Key vocab recap, Day 8 File I/O preview

■ Instructor Note

Strings are the most-used Python type in real projects. Ground every method in a real-world analogy — split() is like cutting text at delimiters, join() is the inverse glue. Regex can feel magic; build patterns letter-by-letter live.

■ PREREQUISITES & PRE-SESSION CHECKLIST

Requirement	Why It Matters
Day 2 — variables & types	Strings are a data type; casting str() used today
Day 4 — loops	Iterating over characters and lines
Day 5 — functions	Students write helper functions for exercises
Day 6 — data structures	split() returns a list; we'll store results in dicts

Pre-Session Checklist	
■ VS Code open, Python extension active	■ Have a few sample strings ready to type live
■ Test: python -c "import re; print('ok')"	■ Whiteboard or screen for drawing regex breakdown
■ Student notebooks distributed / cloned	■ Day 6 recap quiz prepared (3 quick questions)

STEP 1 OF 4 — STRING FORMATTING (0:10 – 0:25, 15 min)

■■ What to Say

"Python has three ways to embed values inside strings — let's see all three."

"The oldest is % formatting — still seen in legacy code."

"Then .format() arrived in Python 3 — more flexible."

"f-strings (Python 3.6+) are what you should use today — fastest and most readable."

"Rule of thumb: always use f-strings in new code. You will see the others in documentation."

Live Code Demo — run in VS Code, students follow along:

```
name = 'Priya'
score = 98.5

# ■■ Method 1: % formatting (old style) ■■
msg1 = "Hello, %s! Your score: %.1f" % (name, score)
print(msg1)
Hello, Priya! Your score: 98.5

# ■■ Method 2: .format() ■■
msg2 = "Hello, {}! Your score: {:.1f}".format(name, score)
print(msg2)
Hello, Priya! Your score: 98.5

# ■■ Method 3: f-strings (preferred) ■■
msg3 = f"Hello, {name}! Your score: {score:.1f}"
print(msg3)
Hello, Priya! Your score: 98.5

# ■■ f-string expressions ■■
print(f"Score rounded: {round(score)}, Pass: {score >= 40}")
Score rounded: 99, Pass: True
```

■ Teaching Tip

Ask students: 'What's different about f-strings?' — they evaluate ANY expression inside {}, not just variables. Demo: f'{2 + 2}', f'{name.upper()}'. This lands well.

Key Vocabulary

Term	Meaning	Example
f-string	Formatted string literal; evaluates {} expressions at runtime	f"Hi {name}"
Format spec	Mini-language inside {} to control display width/precision	:.2f :>10 :,
String literal	A fixed string value written in source code	"hello"

Common Error	Why It Happens	Fix
SyntaxError: f-string expression part cannot include a backslash	Backslash inside {}	Move expression to a variable
ValueError: Single '}' encountered in format string	Literal } not escaped	Use }} to print a literal brace
TypeError: not enough args for format string	% args count mismatch	Count % placeholders vs tuple items

STEP 2 OF 4 — CORE STRING METHODS (0:25 – 0:45, 20 min)

■■ What to Say

"Strings in Python are immutable — every method returns a NEW string; the original never changes."

"Think of these as your text-processing toolkit. You'll use split and join every single day."

"split() breaks a string into a list. join() is the exact reverse — it glues a list into a string."

```
# ■■ split and join ■■
sentence = "  Hello, World! Welcome to Python.  "
words = sentence.split()
print(words)
['Hello,', 'World!', 'Welcome', 'to', 'Python.']
rejoined = " ".join(words)
print(rejoined)
Hello, World! Welcome to Python.

# ■■ strip / lstrip / rstrip ■■
messy = "   Rahul   "
print(messy.strip())
Rahul

# ■■ replace ■■
text = "I love Java. Java is great."
print(text.replace("Java", "Python"))
I love Python. Python is great.

# ■■ upper / lower / title / capitalize ■■
name = "aisha khan"
print(name.title())
Aisha Khan

# ■■ find / count / startswith / endswith ■■
msg = "Python is powerful. Python is fun."
print(msg.count("Python"))
2
print(msg.find("powerful"))
10
print(msg.startswith("Python"))
True
```

Method	Returns	Example
`.split(sep)`	list of str	"a,b,c".split(",") → ['a','b','c']
`sep.join(lst)`	str	" ".join(['a','b']) → 'a b'

.strip()	str	" hi ".strip() → 'hi'
.replace(old,new)	str	"aab".replace("a","x") → 'xxb'
.upper() / .lower()	str	"Hi".lower() → 'hi'
.title()	str	"hello world".title() -> 'Hello World'
.find(sub)	int (-1 if not found)	"hello".find("ll") → 2
.count(sub)	int	"banana".count("a") → 3
.startswith(s)	bool	"Python".startswith("Py") → True
.endswith(s)	bool	"file.py".endswith(".py") → True
.zfill(w)	str	"7".zfill(3) → '007'

■ Done When…

Every student can write: name.strip().title() and explain why .strip() must come first. If stuck, ask: 'What would title() do to leading spaces?'

EXERCISE 1 — Text Cleaner Function (0:45 – 1:00)

```
# Students write this function from the prompt:
# Input: raw user input string (may have extra spaces, mixed case)
# Output: cleaned title-case name, word count, and character count
def clean_text(raw):
    cleaned = raw.strip().title()
    words = cleaned.split()
    return cleaned, len(words), len(cleaned)

name, wc, cc = clean_text("  amir sharma  ")
print(f"Name: {name} | Words: {wc} | Chars: {cc}")
Name: Amir Sharma | Words: 2 | Chars: 10
```

STEP 3 OF 4 — REGEX BASICS WITH re MODULE (1:05 – 1:20, 15 min)

■■ What to Say

"Regular expressions are a mini-language for describing text patterns."

"They look intimidating at first — but we only need 5 patterns today."

"Think of \d as a wildcard for any digit, \w for any word character."

"re.search() finds the first match anywhere in the string."

"re.findall() gives you a list of every match — great for extracting data."

```
import re

# ■■ re.search — find first match ■■
text = "Call us at 98765-43210 or email help@example.com"
match = re.search(r"\d{5}-\d{5}", text)
if match:
    print("Phone found:", match.group())
Phone found: 98765-43210

# ■■ re.findall — all matches ■■
prices = "Items: Rs.250, Rs.1200, Rs.75"
amounts = re.findall(r"\d+", prices)
print(amounts)
['250', '1200', '75']

# ■■ re.sub — replace via pattern ■■
dirty = "  Hello   World  "
clean = re.sub(r"\s+", " ", dirty).strip()
print(repr(clean))
'Hello World'

# ■■ Email validator pattern ■■
email_pattern = r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"
emails = ["user@example.com", "bad-email", "raj@iit.ac.in"]
for e in emails:
    status = "valid" if re.match(email_pattern, e) else "invalid"
    print(f"{e}: {status}")
user@example.com: valid
bad-email: invalid
raj@iit.ac.in: valid
```

Regex Quick-Reference

Pattern	Matches	Example
\d	Any digit 0–9	\d{10} → 10 digits

\w	Word char (letter/digit/_)	\w+ → one or more word chars
\s	Whitespace (space/tab/newline)	\s+ → one or more spaces
.	Any single character	.+ → one or more of anything
^	Start of string	^\d matches leading digit
$	End of string	\d$ matches trailing digit
[abc]	Any one of a, b, c	[aeiou] → any vowel
{n}	Exactly n repetitions	\d{4} → exactly 4 digits
?	0 or 1 (optional)	colou?r → color or colour

■■ Common Regex Pitfall

Always use raw strings (r'pattern') for regex — without r, Python interprets \d as an escape sequence before re even sees it. Teach students to ALWAYS prefix with r.

STEP 4 OF 4 — MINI-PROJECT: CONTACT CARD FORMATTER (1:35 – 1:50, 15 min)

Project Brief

Students build a contact_card() function that takes raw name, phone, and email inputs, cleans and validates them, then prints a formatted card. This combines every concept from today.

```
import re

def contact_card(raw_name, raw_phone, raw_email):
    # ■■ Clean name ■■
    name = raw_name.strip().title()
    # ■■ Validate phone: 10 digits ■■
    digits = re.sub(r"\D", "", raw_phone)
    phone_ok = len(digits) == 10
    phone_fmt = f"{digits[:3]}-{digits[3:6]}-{digits[6:]}" if phone_ok else "INVALID"
    # ■■ Validate email ■■
    pattern = r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"
    email_ok = bool(re.match(pattern, raw_email.strip()))
    email_fmt = raw_email.strip() if email_ok else "INVALID"
    # ■■ Print card ■■
    border = "=" * 35
    print(border)
    print(f"  ■ {name}")
    print(f"  ■ {phone_fmt}")
    print(f"  ✉■  {email_fmt}")
    print(border)

contact_card(" riya mehta ", "9876543210", "riya@example.com")
===================================
  ■ Riya Mehta
  ■ 987-654-3210
  ✉■  riya@example.com
===================================
```

■ Extension Challenge

Fast finishers: add a list parameter for multiple contacts and loop through them, printing a card for each. Or: add a function to extract all emails from a long text blob.

■ SESSION WRAP-UP (1:50 – 2:00)

Student Checklist — Can You Do All of These?

- ■ Create an f-string with number formatting (:.2f, :,)
- ■ Call .strip().title() to clean user input
- ■ Use .split() and ' '.join() to reverse the operation
- ■ Replace substrings with .replace()
- ■ Use re.search() to test whether a pattern exists in a string
- ■ Use re.findall() to extract all matching substrings
- ■ Write a simple email/phone validator with regex
- ■ Build the contact_card() mini-project

■ QUICK REFERENCE CARD — STRINGS & REGEX

String Formatting	String Methods	re Module

`f"{val:.2f}"`
`f"{val:,}"`
`f"{val:>10}"`
	`s.strip()`
	`s.split(",")`
	`",".join(lst)`
	`s.replace(a,b)`
		`import re`
		`re.search(pat,s)`
		`re.findall(pat,s)`
		`re.sub(pat,rep,s)`

■ NEXT SESSION PREVIEW — DAY 08: FILE I/O

Day 08 Topic	File I/O — reading, writing, CSV, JSON
What to tell students	'Next session: we'll make our programs remember things by saving to files.'
String connection	Everything we read from files is a string — today's methods will be used constantly
Key prereq to check	Students must understand context managers (with statement) from today's mini-project

Instructor Troubleshooting Notes

Issue	Solution
Student's regex never matches	Check for missing r prefix on pattern string. Also verify re.match vs re.search.
Confusion: split returns list but join needs string	Show type(s.split()) → list. str.join() takes an iterable.
f-string SyntaxError with backslash	Assign the expression to a variable first, then reference it in f-string.
strip() not removing internal spaces	strip() only trims ends. For internal spaces use .split() + ' '.join() or re.sub.